Sistema de Automatización de Órdenes para Restaurantes mediante Voz e IoT

Documento de diseño y propuesta técnica

Andrick Millan Bernal

Universidad Iberoamericana Ciudad de México

30 de octubre de 2026

Resumen

Este documento presenta el diseño de un sistema de automatización de restaurantes en el que los meseros utilizan un micrófono (dispositivo IoT) para repetir en voz alta la orden del cliente. El audio se transcribe, se interpreta y se envía a un servidor (local o en la nube) que registra el pedido asociado a un número de mesa, lo muestra en tiempo real en una interfaz web (cocina, barra y caja) y genera un ticket virtual. El objetivo principal es **agilizar la toma y el flujo de órdenes**, reduciendo errores y tiempos de espera.

Índice

1. Introducción y problemática

En muchos restaurantes la toma de órdenes sigue siendo manual: el mesero escribe en una libreta, camina hasta la cocina y entrega el papel o lo captura en una terminal. Esto genera:

- Tiempos muertos entre que el cliente ordena y la cocina recibe el pedido.
- Errores de interpretación (letra ilegible, platillos omitidos, mesa equivocada).
- Pérdida de comandas y dificultad para dar seguimiento al estado del pedido.
- Cuentas lentas por la captura manual del ticket final.

La solución propuesta elimina el traslado y la captura manual: el mesero únicamente **dicta la orden** mientras el cliente la realiza y el sistema se encarga del resto.

2. Objetivos

2.1. Objetivo general

Diseñar e implementar un sistema que, a partir de la voz del mesero, registre automáticamente órdenes por mesa, las muestre en una interfaz en tiempo real y genere tickets virtuales, agilizando la operación del restaurante.

2.2. Objetivos específicos

- Capturar la voz del mesero mediante un micrófono conectado a un dispositivo IoT.
- Transcribir e interpretar la orden (mesa, platillos, cantidades y notas).
- Identificar cada mesa con un número único y asociarle sus pedidos.
- Mostrar las órdenes en tiempo real en una UI web para cocina, barra y caja.
- Generar y almacenar un ticket virtual por mesa, con subtotal, impuestos y total.
- Operar sobre un servidor local o en la nube según las necesidades del negocio.

3. Alcance

Dentro del alcance	Fuera del alcance (primera versión)
Dictado de órdenes por voz	Pagos electrónicos integrados (terminal bancaria)
Numeración de mesas y control de estado	Reservaciones en línea para clientes
UI web para cocina, barra y caja	Facturación electrónica (CFDI)
Ticket virtual por mesa	Control completo de inventarios
Panel de administración del menú	Aplicación móvil nativa para clientes

Tabla 1: Alcance del proyecto.

4. Descripción general del sistema

4.1. Flujo de operación

1. El cliente realiza su pedido al mesero.
2. El mesero presiona el botón del micrófono, indica el **número de mesa** y repite la orden.
3. El dispositivo IoT captura el audio y lo envía al servidor.
4. El servidor transcribe la voz a texto (*Speech-to-Text*).
5. El analizador de órdenes (*parser*/NLP) extrae mesa, platillos, cantidades y notas, y los valida contra el menú.
6. La orden se guarda en la base de datos y se emite en tiempo real mediante WebSockets.
7. La UI muestra la comanda en cocina/barra y actualiza el ticket virtual de la mesa.
8. Cocina cambia el estado (*En preparación* → *Lista*) y el mesero es notificado.
9. Al cerrar la mesa, caja cierra el ticket y se registra el pago.

4.2. Ejemplo de dictado

> *"Mesa cuatro: dos tacos al pastor, una orden de guacamole, un agua de horchata y una cerveza sin hielo."*

4.3. Arquitectura general

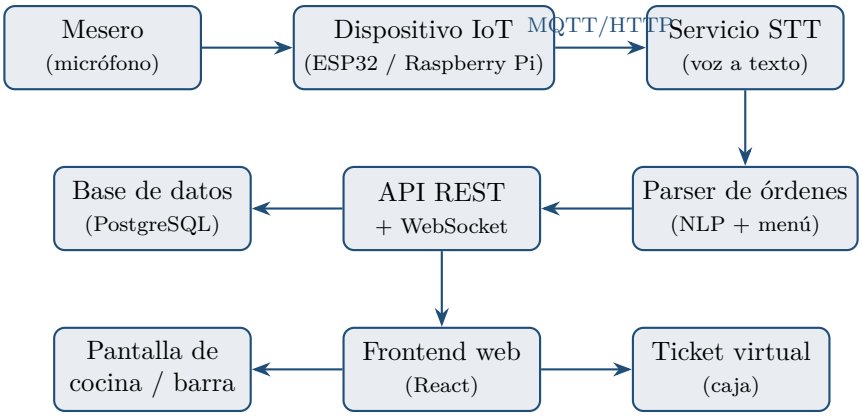


Figura 1: Arquitectura general del sistema.

5. Requerimientos

5.1. Requerimientos funcionales

ID	Descripción	Prioridad
RF-01	El sistema debe capturar audio desde un micrófono del mesero y enviarlo al servidor.	Alta
RF-02	El sistema debe transcribir el audio a texto en español.	Alta

ID	Descripción	Prioridad
RF-03	El sistema debe identificar el número de mesa mencionado en la orden.	Alta
RF-04	El sistema debe reconocer platillos y cantidades y validarlos contra el menú registrado.	Alta
RF-05	El sistema debe permitir al mesero confirmar o corregir la orden interpretada antes de enviarla.	Alta
RF-06	La UI debe mostrar las órdenes en tiempo real, separadas por área (cocina/barra).	Alta
RF-07	Cada mesa debe tener un ticket virtual que se actualice con cada nuevo pedido.	Alta
RF-08	Cocina debe poder cambiar el estado de cada platillo (pendiente, preparando, listo, entregado).	Media
RF-09	El sistema debe permitir cerrar la cuenta de una mesa y registrar el método de pago.	Media
RF-10	El administrador debe poder crear, editar y desactivar platillos y precios.	Media
RF-11	El sistema debe conservar un historial de tickets y órdenes.	Baja

5.2. Requerimientos no funcionales

Atributo	Descripción
Rendimiento	Latencia total (fin del dictado a visualización en pantalla) menor a 3 segundos.
Disponibilidad	Operación local ante falta de Internet (modo *offline-first* con servidor local).
Precisión	Reconocimiento de órdenes igual o superior al 90 % con confirmación del mesero.
Seguridad	Autenticación por roles, comunicación cifrada (TLS) y dispositivos IoT autenticados.
Usabilidad	Interfaz clara, botones grandes y uso con una sola mano.
Escalabilidad	Soporte para múltiples meseros, mesas y sucursales.

Tabla 3: Requerimientos no funcionales.

6. Componentes del sistema

6.1. Capa IoT (captura de voz)

Cada mesero porta un dispositivo con micrófono y un botón *push-to-talk*. El dispositivo graba el audio mientras el botón está presionado y lo envía al servidor.

Opción	Ventajas	Desventajas
ESP32 + micrófono I2S (INMP441)	Muy económico, bajo consumo, Wi-Fi integrado	Procesamiento limitado; requiere STT externo
Raspberry Pi + micrófono USB	Mayor capacidad, permite STT local	Mayor costo y consumo
Teléfono / tablet con la web app	Sin hardware adicional	Menos ergonómico; depende del dispositivo del mesero

Tabla 4: Opciones de hardware para la captura de voz.

6.2. Procesamiento de voz y de órdenes

- **Speech-to-Text:** Whisper (local o API), Google Speech-to-Text o Vosk (offline).
- **Parser de órdenes:** reglas y expresiones regulares para número de mesa y cantidades, más coincidencia aproximada (*fuzzy matching*) contra el menú. Opcionalmente un modelo de lenguaje para casos ambiguos.
- **Confirmación:** la orden interpretada se muestra al mesero (pantalla o aplicación) antes de enviarse a cocina.

6.3. Backend

Elemento	Propuesta
Lenguaje / framework	Node.js (Express o NestJS) o Python (FastAPI)
Comunicación IoT	MQTT (Mosquitto) o HTTP/HTTPS
Tiempo real	WebSockets (Socket.IO)
Base de datos	PostgreSQL (alternativa: MongoDB)
Autenticación	JWT con roles (mesero, cocina, caja, administrador)
Despliegue	Servidor local (mini-PC / Raspberry Pi) o nube (AWS, Azure, Render) con Docker

Tabla 5: Tecnologías del backend.

6.4. Frontend (UI)

Aplicación web responsiva (React, Vue o similar) con las siguientes vistas:

7. Identificación de mesas y ticket virtual

Cada mesa se registra con un **número único** (por ejemplo 1 a N). Al dictarse una orden, el número de mesa se usa como llave para abrir (o actualizar) el ticket asociado. Mientras la mesa esté ocupada existe un único ticket *abierto*; al cerrar la cuenta pasa a estado *cerrado*.

Vista	Función
Mapa de mesas	Muestra todas las mesas numeradas y su estado (libre, ocupada, cuenta solicitada).
Cocina / Barra	Tablero tipo Kanban con comandas nuevas, en preparación y listas.
Ticket de mesa	Detalle de platillos, cantidades, notas, subtotal, impuestos y total.
Caja	Cierre de cuenta, método de pago e historial.
Administración	Gestión de menú, precios, mesas y usuarios.

Tabla 6: Vistas de la interfaz.

7.1. Estados de una mesa

Estado	Descripción
Libre	Sin ticket abierto.
Ocupada	Ticket abierto con pedidos activos.
Cuenta solicitada	El cliente pidió la cuenta; no se admiten nuevos pedidos sin reabrir.
Cerrada	Ticket pagado y archivado.

Tabla 7: Estados de las mesas.

7.2. Contenido del ticket virtual

- Folio, fecha y hora de apertura y cierre.
- Número de mesa y mesero responsable.
- Lista de platillos: cantidad, nombre, notas, precio unitario e importe.
- Subtotal, impuestos (IVA), propina sugerida y total.
- Estado de cada platillo y método de pago.

7.3. Ejemplo de ticket

8. Modelo de datos

Listing 1: Fragmento del esquema SQL (PostgreSQL).

```
CREATE TABLE mesas (
  id        SERIAL PRIMARY KEY,
  numero    INT UNIQUE NOT NULL,
  capacidad INT DEFAULT 4,
  estado    VARCHAR(20) DEFAULT 'libre'
);

CREATE TABLE tickets (
  id         SERIAL PRIMARY KEY,
  mesa_id    INT REFERENCES mesas(id),
  mesero_id  INT REFERENCES usuarios(id),
```

TICKET #0001 — MESA 4

Mesero: Juan P. Fecha: 30/10/2026 16:45

Cant.	Platillo	P. Unit.	Importe
2	Tacos al pastor	$80.00	$160.00
1	Guacamole	$95.00	$95.00
1	Agua de horchata	$35.00	$35.00
1	Cerveza (sin hielo)	$55.00	$55.00
		Subtotal	$345.00
		IVA (16 %)	$55.20
		Total	**$400.20**

Tabla 8: Ejemplo ilustrativo de ticket virtual.

Tabla	Campos principales
`mesas`	id, numero, capacidad, estado
`usuarios`	id, nombre, rol, contrasena_hash
`platillos`	id, nombre, categoria, precio, area (cocina/barra), activo
`tickets`	id, mesa_id, mesero_id, estado, abierto_en, cerrado_en, subtotal, impuestos, total
`ordenes`	id, ticket_id, texto_original, creada_en
`orden_items`	id, orden_id, platillo_id, cantidad, notas, estado
`pagos`	id, ticket_id, metodo, monto, fecha

Tabla 9: Entidades de la base de datos.

```
  estado      VARCHAR(20) DEFAULT 'abierto',
  abierto_en  TIMESTAMP DEFAULT NOW(),
  cerrado_en  TIMESTAMP,
  subtotal    NUMERIC(10,2) DEFAULT 0,
  impuestos   NUMERIC(10,2) DEFAULT 0,
  total       NUMERIC(10,2) DEFAULT 0
);

CREATE TABLE orden_items (
  id          SERIAL PRIMARY KEY,
  orden_id    INT REFERENCES ordenes(id),
  platillo_id INT REFERENCES platillos(id),
  cantidad    INT NOT NULL,
  notas       TEXT,
  estado      VARCHAR(20) DEFAULT 'pendiente'
);
```

9. API del backend

Método	Endpoint	Descripción
POST	`/api/voz`	Recibe el audio del dispositivo IoT y devuelve la orden interpretada.
POST	`/api/ordenes`	Confirma y registra una orden en el ticket de la mesa.
GET	`/api/mesas`	Lista las mesas y sus estados.
GET	`/api/tickets/:mesa`	Obtiene el ticket abierto de una mesa.
PATCH	`/api/items/:id`	Actualiza el estado de un platillo.
POST	`/api/tickets/:id/cerrar`	Cierra la cuenta y registra el pago.
GET/POST	`/api/menu`	Consulta y administra el menú.

Tabla 10: Endpoints principales.

Listing 2: Ejemplo simplificado del parser de órdenes (Python).

```python
import re

NUMEROS = {"un": 1, "una": 1, "uno": 1, "dos": 2, "tres": 3,
           "cuatro": 4, "cinco": 5, "seis": 6}

def interpretar(texto, menu):
    texto = texto.lower()
    mesa = re.search(r"mesa\s+(\w+)", texto)
    num_mesa = NUMEROS.get(mesa.group(1), None) if mesa else None
    items = []
    for platillo in menu:
        if platillo.lower() in texto:
            items.append({"platillo": platillo, "cantidad": 1})
    return {"mesa": num_mesa, "items": items}
```

10. Seguridad y roles

Rol	Permisos
Mesero	Dictar órdenes, confirmar/corregir, ver sus mesas y tickets.
Cocina / Barra	Ver comandas de su área y cambiar el estado de los platillos.
Caja	Consultar tickets, cerrar cuentas y registrar pagos.
Administrador	Gestionar menú, usuarios, mesas y reportes.

Tabla 11: Roles del sistema.

Se recomienda cifrar la comunicación con TLS, autenticar cada dispositivo IoT con credenciales únicas y almacenar contraseñas con *hash* (bcrypt).

Criterio	Servidor local	Nube
Latencia	Muy baja (red del restaurante)	Depende de Internet
Disponibilidad sin Internet	Sí	No (requiere conexión)
Costo inicial	Mayor (hardware)	Bajo
Mantenimiento	Responsabilidad del negocio	Gestionado por el proveedor
Multi-sucursal	Más complejo	Sencillo

Tabla 12: Comparativa de despliegue.

11. Servidor local vs. nube

Recomendación: iniciar con un servidor local (mini-PC o Raspberry Pi) para garantizar baja latencia y operación sin Internet, con sincronización opcional a la nube para respaldos y reportes.

12. Plan de trabajo

Fase	Actividades	Duración
1. Análisis	Levantamiento de requerimientos y definición del menú.	1 semana
2. Diseño	Arquitectura, base de datos y mockups de la UI.	2 semanas
3. Backend	API, base de datos, WebSockets y parser.	3 semanas
4. IoT	Prototipo de micrófono y envío de audio.	2 semanas
5. Frontend	Vistas de mesas, cocina, ticket y administración.	3 semanas
6. Integración y pruebas	Pruebas de voz, latencia y usabilidad en campo.	2 semanas
7. Despliegue	Instalación, capacitación y ajustes.	1 semana

Tabla 13: Cronograma estimado (aprox. 14 semanas).

13. Riesgos y mitigación

14. Conclusiones

El sistema propuesto integra IoT, procesamiento de voz, un backend en tiempo real y una interfaz web para reducir el tiempo entre la orden del cliente y la preparación del platillo. La identificación de mesas por número y el ticket virtual permiten un control claro de cada cuenta, disminuyen los errores y facilitan el trabajo de meseros, cocina y caja. La arquitectura modular permite empezar con un prototipo sencillo y escalar progresivamente hacia múltiples sucursales.

Riesgo	Mitigación
Ruido ambiental afecta el reconocimiento de voz	Micrófonos con cancelación de ruido y modo *push-to-talk*.
Interpretación errónea de platillos	Paso de confirmación del mesero y vocabulario restringido al menú.
Caída de Internet	Servidor local y modo offline.
Resistencia del personal al cambio	Capacitación y periodo de prueba con respaldo manual.
Fallo del dispositivo IoT	Dispositivos de repuesto y alternativa mediante la web app.

Tabla 14: Riesgos identificados.